

Modifiche IPA — AOT-literal universale & Input Vector flessibile

Refactor del design space P1/P2/P3 · documento cronologico delle modifiche (Task 1–11) · 2026

Stato di verifica. I Task 1–7 sono stati scritti su Windows e verificati a livello Python (sintassi, generazione sorgenti, byte-compatibilità); i Task 8, 9 e 11 sono stati **eseguiti e verificati su Linux/Kathara** (`test_suite --only kernel` PASS, deploy AOT attaccato, benchmark misurati). L'unico punto ancora **da rimisurare su Linux** è il Task 10 (`mac_table` → ARRAY): il codice è scritto e compila, ma i numeri della tabella comparativa vanno rifatti perché raccolti con la vecchia mappa hash. La sviluppo resta su Windows, dove eBPF/BCC/clang-bpf/verifier non girano.

Sommario delle attività

#	Attività	Esito
1	AOT-literal universale per hardcoded (P1)	FATTO
2	AOT “semplice” per template/modular	SALTATO
3	Input Vector flessibile a runtime per template (P2) e modular (P3)	FATTO
4	Pulizia codice obsoleto	FATTO
5	Polling MAC per-classe — correzione robustezza	FATTO
6	AOT-literal in sostituzione di BCC per il deploy hardcoded	FATTO
7	Profondità variabile in P1 hardcoded	FATTO
8	Trade-off larghezza vs profondità — misurato	FATTO — Linux
9	Loader AOT fully-static (fix GLIBC) — deploy verificato su Kathara	FATTO — Linux
10	<code>mac_table</code> BPF_HASH → BPF_ARRAY (tutte le pipeline)	FATTO (da rimisurare)
11	Robustezza test + pulizia (min-di-N, alt-arch, dead code, test rimossi)	FATTO

Il concetto chiave che ha guidato le scelte. Due assi *ortogonali*: **pesi** (literal in P1 · in mappa in P2/P3) e **compilazione** (AOT offline · BCC a runtime). “AOT-literal” = literal ∧ AOT, specifico di P1. Non è applicabile ai map-based senza snaturarli (Task 2). Invece la **flessibilità dell'IV** (descrittore per-modello) è ortogonale e si porta ovunque (Task 3).

Task 1 — AOT-literal universale (hardcoded P1)

FATTO

Problema di partenza

Il generatore AOT offline (`gen_full_c.py` , dialetto libbpf) era **cablato** al solo descrittore default `65-4-4-7` .
Qualsiasi descrittore sparse/eterogeneo era supportato solo dal path BCC (`ebpf_program.py`), non dall'AOT.
Un guard rifiutava i descrittori non-default.

Cosa è cambiato

`gen_full_c.py` è ora **descrittore-driven**: i 3 *kind* di feature sono stati portati dal dialetto BCC al dialetto libbpf, riproducendo la stessa logica di `ebpf_program._gen_feature_*` :

```
scalar          -> ttl = ip->ttl                                (dal pacchetto)
dense_vector_map -> ls0..lsN via 1 bpf_map_lookup_elem        (link_state / queue_state)
onehot          -> un solo switch che seleziona 1 peso        (ingress_iface / node)
```

Ogni descrittore accettato dal BCC è ora anche **AOT-compilabile**. Le mappe dense (`link_state` , `queue_state`) vengono emesse solo se il descrittore le usa, dimensionate dalla topologia.

File modificati

File	Modifica
<code>shared/poc_aot/gen_full_c.py</code>	Riscritto descrittore-driven: <code>_feat_scalar/_feat_dense_vector/_feat_onehot_*</code> , <code>_emit_maps()</code> per-descrittore, <code>_resolve_shape()</code> via <code>model_meta.derive_shape</code> .
<code>shared/methods/method4_hardcoded_aot.py</code>	Rimosso il guard <code>_check_supported / _SUPPORTED</code> ; passa <code>meta + topology_config</code> al generatore.
<code>shared/poc_aot/loader_aot.c</code>	Seeding generico: <code>seed_vec_map()</code> semina <code>link_state / queue_state</code> solo se presenti (dimensione dalla mappa); classi <code>mac_table</code> derivate da <code>n_out</code> .

Come funziona adesso

- **Default** [`link_state`, `ingress_iface`, `ttl`, `node`] → `65-4-4-7` **byte-identico** a prima (stessi pesi, stesse colonne; ora combacia col dialetto BCC).
- **Custom** es. [`link_state`, `queue_occupancy`, `ttl`] → `n_in=11` , emette mappa `queue_state` , `qs0..qs3` , niente `node/iface`.
- Il path **BCC resta intatto** come fallback garantito.

Verificato (Python). Diff generatore nuovo vs vecchio sul default: solo differenze cosmetiche + riordino commutativo dei termini, *pesi e colonne identici*. Descrittore custom: mappa `queue_state` emessa, slicing pesi corretto (bias a offset 44 per 11-4-4-7).

Task 2 — AOT “semplice” per template/modular SALTATO

Sconsigliato e non implementato. Motivo: **beneficio marginale per design**. Template e modular compilano **una sola volta** all'avvio (il programma è indipendente dal modello; nuovo modello = solo `bpf_map_update_elem`). L'AOT toglierebbe una singola compilazione di startup, non i “1660 ms per modello” che affliggono solo P1. In più richiederebbe riscrivere due programmi interi dal dialetto BCC a libbpf. Costo alto, ritorno quasi nullo.

Task 3 — Input Vector flessibile a runtime (P2 template, P3 modular) FATTO

Problema di partenza

Le pipeline map-based avevano l'IV **cablato** al layout protocollare a 65 (link_state 0-5, iface one-hot 6-11, ttl 12, node one-hot 13-64), con offset fissi nel C. I pesi erano già in mappa (nessuna ricompilazione per modello) ma il *layout delle feature* era rigido: tutti i modelli dovevano usare le stesse 4 feature nello stesso ordine.

Decisioni di design (confermate)

Decisione	Scelta
Sorgente del descrittore a runtime	Registry map <code>model_desc[model_id]</code> , popolata dal control plane. NON dal pacchetto (autorevole, niente fiducia nel traffico, zero costo/pacchetto extra oltre 1 lookup).
Ceilings di unroll	<code>MAX_FEAT=4</code> . Le dimensioni dense sono topology-fixed (link_state=6, queue=4); i one-hot grandi (node 52) sono un <i>singolo lookup runtime-indexed</i> , non srotolati.
Tecnica di loop	<code>#pragma unroll</code> (portabile, stile esistente; nessuna dipendenza da kernel ≥ 5.13).
Coppie <code>feat*</code> dell'header IPA	Restano metadata/osservabilità , NON sorgente dell'IV.

La nuova registry `model_desc`

```
#define MAX_FEAT 4
struct feat_ent { __u8 code; __u8 size; __u8 col_off; __u8 _pad; };
struct model_desc { __u8 n_feat; __u8 n_in; __u8 p0; __u8 p1;
                    struct feat_ent feats[MAX_FEAT]; }; /* 20 byte */
BPF_HASH(model_desc, __u8 model_id, struct model_desc, 256);
```

`col_off` = colonna iniziale della feature nella riga di input di `fc1` (somma cumulativa delle `size` precedenti). Il control plane la calcola con `model_meta.resolve_descriptor()`, così l'IV a runtime coincide col layout dei pesi allenati.

Il primo layer, ora descrittore-driven

Gli offset cablati (`+12 ttl`, `+13 node`, `5+iface`) sono sostituiti da un loop unrolled su `MAX_FEAT` che, per ogni neurone `j`, accumula il contributo di ogni feature dichiarata alla sua colonna runtime:

```
for (f = 0; f < MAX_FEAT; f++) if (f < desc->n_feat) {
    base = woff + fc1_w_off + j*n_in + desc->feats[f].col_off;
    switch (code) {
        TTL          -> acc += _ttl * W(base);
        LINK_STATE    -> for i<size: acc += ls[i] * W(base+i);    // unroll 6
        QUEUE_OCC      -> for i<size: acc += qs[i] * W(base+i);    // unroll 4
        INGRESS_IF     -> if active: acc += W(base + (iface-1));    // 1 lookup
        NODE           -> if node<size: acc += W(base + node);    // 1 lookup
    }
}
```

`n_in` è ora runtime (da `model_desc`) e guida il layout dei pesi (`fc1_b_off = n_in*n_h1`, ecc.). Gli indici restano *bounded* (il check `idx >= MAX_WEIGHT_ENTRIES` / il `lookup` che ritorna NULL) → verifier-safe.

File modificati

<code>shared/model_meta.py</code>	Aggiunti <code>FEATURE_CODE</code> (codici numerici, sorgente unica: <code>link_state=1, iface=2, ttl=3, node=4, queue=5</code>) e <code>resolve_descriptor()</code> (→ lista <code>{code,size,col_off}</code>).
<code>shared/ebpf_template_arch.py</code>	Mappe <code>model_desc + queue_state</code> ; <code>fc1</code> descrittore-driven; <code>n_in</code> runtime nel layout pesi; <code>arch_weight_count(n_h1,n_h2,n_in)</code> parametrizzato; <code>load_model_desc()</code> ; <code>load_arch_weights()</code> semina <code>model_desc</code> (default) internamente.
<code>shared/ebpf_modular.py</code>	Stesse mappe/strutture nel common header; <code>layer_first</code> descrittore-driven; check first-layer <code>n_in==65</code> rilassato a <code>[1,128]</code> ; <code>load_model_desc()</code> ; <code>load_modular_weights()</code> semina <code>model_desc</code> internamente.
<code>shared/methods/method5_template.py</code> , <code>method6_modular.py</code>	Nessuna chiamata extra necessaria (il seeding è integrato nei loader dei pesi).

Verificato (Python). Il descrittore default risolve a `[(1,6,0),(2,6,6),(3,1,12),(4,52,13)]`, `n_in 65` → **esattamente** gli offset cablati vecchi (comportamento byte-identico). Descrittore custom `[link_state,queue,ttl]` → `col_off 0/6/10`. Struct `model_desc` = 20 byte (combacia col C). Conteggio mappe: template 2/2 (dispatcher + ramo standalone), modular 1/1 (common header incluso una volta). Zero offset cablati residui.

Compatibilità test harness. Il seeding di `model_desc` è stato **integrato dentro** `load_arch_weights()` e `load_modular_weights()`: così *tutti* i chiamanti esistenti (i metodi e i test: `verify_prog_run`, `verify_multi_model`, `bench_model_add`, `test_suite`) registrano il descrittore default senza modifiche. Senza questo, l'inferenza avrebbe restituito `XDP_PASS` (lookup fallito).

Nota sulla narrativa design-space

P2/P3 diventano **descrittore-flessibili** come P1: non più "65 fisso". Da concordare col relatore come impatta il confronto della tesi.

Task 4 — Pulizia obsoleti FATTO

Codebase risultato pulito. Eliminati 3 alias di retrocompatibilità **morti** (0 usi) in `shared/model_meta.py`, residui del refactor `node_config` → `topology_config`:

- `DEFAULT_NODE_CONFIG` (alias di `DEFAULT_TOPOLOGY_CONFIG`)
- `load_node_config` (alias di `load_topology_config`)
- `node_config_for` (alias di `topology_config_for`)

Il resto (`method4/5/6`, `test_ipa`, `fix_bpf.sh`, `host_headers`, `weights_method2.json`) è attivo e conservato. Rimosso anche il guard obsoleto `_check_supported` dell'AOT (parte del Task 1).

Task 5 — Polling MAC per-classe: correzione FATTO

Problema di partenza

`install_mac_per_class()` (già introdotta per far sì che ogni classe dell'argmax usi il MAC risolto via ARP della **propria** porta egress, non sempre la stessa) aveva due difetti scoperti in revisione:

- **Doppia lettura ARP disallineata:** la funzione risolveva il MAC del vicino internamente, poi ciascun metodo (`method4/5/6`) rifaceva *una seconda* lettura di `/proc/net/arp` per decidere quali classi avviare in polling di refresh. Tra le due letture l'ARP poteva risolversi → il thread di refresh non veniva avviato per una classe in realtà ancora sul MAC di fallback, oppure veniva avviato per una classe già corretta.
- **Thread di refresh che si auto-terminava:** `start_mac_refresh_thread` usciva dal loop non appena tutte le classi avevano risolto una volta. Ma una entry ARP può scadere o il vicino dietro una porta può cambiare (flap, riavvio) — dopo l'uscita del thread nessuno correggeva più la entry nella mappa BPF.

Cosa è cambiato

`install_mac_per_class()` ora calcola risoluzione ARP e "quali classi restano in fallback" nella **stessa, unica lettura**, e ritorna `{"installed": [...], "pending": [(cls, iface), ...]}`. I tre metodi passano `pending` direttamente a `start_mac_refresh_thread`, senza ricalcolarlo. Il thread di refresh ora gira per **tutta la vita**

della pipeline (non esce più al primo giro completo), riscrivendo la entry BPF solo quando il MAC risolto cambia rispetto all'ultimo scritto — quindi in stato stazionario è di sola lettura, nessun overhead di scrittura extra.

File modificati

File	Modifica
shared/common.py	install_mac_per_class() ritorna {"installed","pending"} da un'unica lettura ARP; start_mac_refresh_thread() gira indefinitamente, scrive solo sui cambiamenti.
shared/methods/method4_hardcoded.py , method5_template.py , method6_modular.py	Consumano mac_info["pending"] invece di ricalcolare con una seconda neighbor_mac() .

Verificato (Linux, Kathara). Il comportamento per-classe (class 0 -> eth0 , class 1 -> eth1 , ecc., ARP-risolto) era già confermato funzionante prima di questa correzione; la correzione riguarda solo la **robustezza** del polling (niente più doppia lettura disallineata, niente più thread che smette di sorvegliare). Consigliato un nuovo giro di fumo su Kathara dopo l'update, dato che la firma di ritorno è cambiata.

Task 6 — AOT-literal in sostituzione di BCC per il deploy hardcoded FATTO

Richiesta esplicita del relatore: AOT-literal deve **sostituire** BCC come backend di deploy per hardcoded, non restare una semplice alternativa. Fatto. Il vincolo che inizialmente sembrava bloccante — i nodi Kathara non hanno clang /libbpf (FileNotFoundError: clang) — riguardava solo la build (che infatti resta offline, su build-box) e il loader (loader_aot.c), che era linkato **dinamicamente** contro libbpf: un binario compilato altrove non sarebbe comunque partito su un nodo senza libbpf.so .

Fix: loader_aot.c ora si linka **staticamente** contro libbpf (+ libelf/zlib, sue dipendenze), lasciando dinamica solo libc — binario autosufficiente, buildato una volta su una build-box, eseguibile su qualunque nodo Kathara senza clang/libbpf installati. execute_pipeline.py non offre più la scelta --hardcoded-backend : AOT-literal è l'unico percorso di deploy per hardcoded.

Eccezione mantenuta, deliberata: l'infrastruttura di test interna (verify_prog_run.py , test_suite.py --only kernel) continua a usare BCC per compilare-e-verificare — non gira mai sul nodo datapath in produzione, è un problema distinto dal deploy.

Task 7 — Profondità variabile in P1 hardcoded FATTO

Problema di partenza

Sia il generatore BCC (`ebpf_program.generate_ebpf_hardcoded`) sia quello AOT (`gen_full_c._emit_model_body`) avevano la profondità **cablata a 2 hidden layer** (`n_h1`, `n_h2` = `hidden_dims`). Anche il riferimento Python usato dal kernel test (`verify_prog_run.ref_infer_sparse`) assumeva esattamente 2 layer.

Cosa è cambiato

`hidden_dims` è ora una sequenza di **lunghezza qualsiasi**: (4,4) (storico), (8,) (1 layer), (4,4,4,4) (4 layer), () (modello lineare puro, nessun hidden layer). Layout dei pesi generico per-layer (`n_prev*n_cur` pesi poi `n_cur` bias, layer per layer), che si riduce esattamente al vecchio schema a 2 layer quando `hidden_dims == (n_h1, n_h2)` .

File modificati

File	Modifica
<code>shared/ebpf_program.py</code>	<code>generate_ebpf_hardcoded</code> : <code>layer_sizes</code> = <code>[n_in] + dims + [n_out]</code> , loop generico sui layer invece di <code>fc1/fc2/out</code> cablati; gestisce anche 0 hidden layer (il primo layer diventa direttamente l'output, niente ReLU).
<code>shared/poc_aot/gen_full_c.py</code>	Stessa generalizzazione lato AOT: <code>_layer_sizes()</code> , <code>_weight_count()</code> , <code>_slices()</code> parametrici sul numero di layer.
<code>shared/test/verify_prog_run.py</code>	<code>ref_infer_sparse</code> : stesso schema generico per-layer lato riferimento Python (il PASS/FAIL del kernel test ora funziona per qualunque profondità, non solo 2).
<code>shared/methods/method4_hardcoded_aot.py</code>	Bonus: corretto un <code>NameError</code> latente (<code>build_ms</code> referenziato non definito nel path "clang assente, riuso <code>.o</code> prebuilt" senza <code>--iface</code>).

Verificato (Python). Output del generatore BCC per `hidden_dims=(4,4)` confrontato byte-per-byte con la versione precedente al refactor: identico (a meno di righe vuote). Riferimento Python generalizzato confrontato numericamente contro la vecchia implementazione esplicita a 2 layer su pesi/input casuali: risultato identico (stessa classe, stesso valore). `model_meta.json` 's `hidden_dims` (già letto ovunque) accetta ora qualunque profondità senza altre modifiche.

Task 8 — Trade-off larghezza vs profondità (P1 hardcoded)

FATTO — misurato su Linux

Domanda

A parità di budget di pesi, conviene aumentare i neuroni di un layer (**largo**) o aggiungere un nuovo layer (**profondo**)? Nuovo script: `shared/test/bench_depth_vs_width.py` .

Metodologia

- **3 tier a budget-pesi abbinato** (A ~300, B ~1200, C ~4700 pesi), ciascuno con una forma larga (1 layer) e due profonde (4 e 8 layer), scarto di pesi dichiarato esplicitamente (mai assunto "circa uguale").
- **4 descrittori di feature diversi** (0, 1, 2 feature one-hot; one-hot piccola vs grande) per isolare l'effetto del descrittore da un effetto generale larghezza/profondità — il descrittore di default ha una feature one-hot molto costosa (`node` , size 52) che avrebbe potuto falsare la conclusione se testata da sola.
- **Statistica: minimo su 7 trial indipendenti**, non media/mediana di un solo campione. Un run preliminare con un solo campione (`repeat=2000`) mostrava oscillazioni fino a 20× senza alcuna correlazione col numero di istruzioni — rumore di sistema **a senso unico** (un interrupt/scheduling può solo rallentare un trial, mai accelerarlo), quindi il minimo è la statistica corretta per stimare il costo al netto delle interferenze (stesso principio di hyperfine / Google Benchmark).
- **Isolamento in subprocess**: quando una forma sfiora lo stack eBPF di 512 byte, il backend LLVM di BCC termina con un abort **fatale, non catturabile** come eccezione Python. Due tentativi di stimare a priori la soglia di overflow con una formula sono stati smentiti dal comportamento reale del compilatore (-O2 a volte riesce a riutilizzare slot di stack, a volte no, in modo non prevedibile). Soluzione: ogni cella (descrittore × forma) gira nel suo **proprio subprocess** — un crash marca solo quella cella e lo sweep prosegue.

Risultato (minimo su 7 trial, 4 descrittori, box Linux)

Tier	Forma	Pesi	ns/pkt (min, range sui 4 descrittori)
A (~100-320 pesi)	baseline / wide 1 layer	~90-320	44 - 56 ns
	deep 8×3	~150-310	57 - 76 ns (overhead profondità: +13/+32 ns)
B (~300-1300 pesi)	wide 1×16	~310-1175	103 - 111 ns (sempre il più veloce)
	deep 4×11	~610-1210	203 - 236 ns (~2× più lento)
	deep 8×9	~810-1295	269 - 301 ns (~2.5-3× più lento)
C (~1200-4700 pesi)	tutte (wide/deep 4/deep 8)	—	CRASH sempre , ogni descrittore, ogni forma

Cosa significa

A parità di budget-pesi, allargare batte approfondire — risultato coerente su tutti e 4 i descrittori indipendenti testati (non un artefatto delle feature one-hot del descrittore di default). Ogni layer nascosto in più costa un overhead fisso (~15-40 ns/layer, transizione + ReLU) indipendente dalla composizione delle feature. Al budget più piccolo (Tier A) la differenza è presente ma contenuta; al budget medio (Tier B) diventa un fattore 2-3×.

Tetto di capacità assoluto: oltre ~1200-1300 pesi lo stack eBPF (512 byte) va in overflow **sempre**, larga o profonda che sia la rete — non è una scelta di design larghezza/profondità, è un limite strutturale dell'architettura "tutto srotolato in un'unica funzione C, pesi come literal". Per modelli più grandi serve spostare gli array grandi in una `BPF per-cpu array map` (suggerimento diretto del compilatore nel messaggio di errore), non più semplicemente redistribuire gli stessi pesi su più layer.

Come rilanciare

```
sudo python3 shared/test/bench_depth_vs_width.py           # tutti i 4 descrittori
sudo python3 shared/test/bench_depth_vs_width.py --descriptor no_onehot
sudo python3 shared/test/bench_depth_vs_width.py --repeat 5000 # più stabile, più lento
```

Task 9 — Loader AOT fully-static: fix GLIBC e deploy verificato

FATTO — verificato su Kathara

Estende il Task 6. Il link **statico di libbpf** non bastava: il loader restava collegato **dinamicamente a glibc**, e un binario glibc non è retrocompatibile — compilato su host recente (glibc ≥ 2.38) falliva sul nodo Kathara con glibc più vecchia (`version 'GLIBC_2.38' not found`).

Fix: link **completamente statico** (`-static` , glibc inclusa). Il loader fa solo syscall bpf + I/O su file, niente risoluzione hostname/dlopen, quindi fully-static è sicuro. Lo script prova progressivamente più librerie statiche (`libbpf.a libelf.a libz.a` → + `libzstd.a liblzma.a` → + `libbz2.a` , perché elfutils recente comprime le sezioni ELF), e se il `-static` fallisce stampa l'errore `ld` completo e avvisa che il binario glibc-dinamico può non partire su nodi con glibc più vecchia. Gestiti anche i casi "no `cc` /gcc sul nodo" (build va fatta sull'host, il binario statico è visibile via bind-mount) e il `PermissionError` da file generati come root (`chown -R $USER`).

Verificato end-to-end su Kathara (frankfurt). `built loader_aot (fully static: ...) ; deploy con execute_pipeline.py --method hardcoded --iface eth1 ; ip link show dev eth1` mostra `prog/xdp id ... name xdp_dispatch ... jited`. Nodo senza `clang` , `cc` né `libbpf` usabile per il link → la sostituzione BCC→AOT per il deploy è dimostrata. (L' `-EPERM` del bench sull'host senza root è atteso, non un errore del deploy.)

Task 10 — `mac_table` : BPF_HASH → BPF_ARRAY

FATTO (codice) — da rimisurare su Linux

Problema

Collo di bottiglia **trasversale** (identico in P1/P2/P3, non legato alla scelta pipeline). `mac_table` traduce la classe dell'argmax (intero denso 0..6) nell'azione di forwarding, ma era dichiarata `BPF_HASH` : ogni pacchetto redirettato paga hash + bucket + confronto chiave, inutile con chiavi dense.

Cosa è cambiato

`BPF_ARRAY` : indice diretto O(1), memoria contigua, niente hash. Applicato a `mac_table` , `mac_table_t2` , `mac_table_t3` , al generatore AOT (`gen_full_c` , `BPF_MAP_TYPE_ARRAY`) e ai programmi baseline dei test.

Punto critico gestito. Una `BPF_ARRAY` **non ritorna mai NULL** (slot sempre presenti, azzerati). La logica "classe non provvista → MISS" non può più basarsi su `action == NULL` : ora usa `action->ifindex != 0` come sentinella (ifindex 0 non è mai un'uscita valida). Cambiato in tutti i punti d'azione delle 3 pipeline + AOT + baseline.

File modificati

<code>shared/ebpf_program.py</code>	dichiarazione + check epilogo (P1)
<code>shared/ebpf_template_arch.py</code> , <code>ebpf_modular.py</code>	<code>mac_table_t2/t3</code> ARRAY + check <code>ifindex != 0</code>
<code>shared/poc_aot/gen_full_c.py</code>	SEC(".maps") tipo ARRAY + check <code>act->ifindex != 0</code>
<code>shared/test/verify_prog_run.py</code>	programmi baseline (EBPF_BASELINE, EBPF_BASELINE_TAILCALL) allineati ad ARRAY

Da rimisurare. I numeri della tabella comparativa sono stati raccolti con HASH; una rimisurazione (`test_suite --only kernel`) mostrerebbe un piccolo ulteriore miglioramento sul redirect. Il seeder userspace non cambia (le classi non scritte restano azzerate → ifindex 0 → MISS).

Task 11 — Robustezza dei test e pulizia FATTO

- **Metodologia di misura corretta (min-di-N):** `test_suite.py --only kernel` misurava latenza/throughput con un *singolo* campione `BPF_PROG_TEST_RUN` , che oscillava anche 2-5× tra run (rumore a senso unico). Ora gira ciascuna pipeline 7 volte e riporta il **minimo** (+ p50/max), stessa correzione già fatta in `bench_depth_vs_width.py` . Flag `--kernel-trials` .
- **Verifica architetture alternative nel kernel test:** `test_suite --only kernel` ora verifica anche shape diverse dal 65-4-4-7 (hardcoded a 1 e 3 hidden layer; template 65-6-5-7 e modular 65-5-6-4-7 registrati insieme al reale). Prima copriva una sola architettura.
- **Nuovo** `bench_tailcall_overhead.py` : isola il costo puro di *una* tail call (baseline con/senza un hop, stesso parse e stessa azione), scorporandolo da MLP e doppio parsing.
- **Polling link_state a 0.5 s:** `method5` / `method6` sovrascrivevano il default con `interval=1.0` — riportati a 0.5 s, coerenti col default.
- **Dead code rimosso:** `method4_hardcoded.py` era diventato solo verify-only (deploy è AOT), ma conteneva ancora `_attach` (live-attach BCC) mai più invocato → rimosso; il file è ora esplicitamente il path BCC `compile-and-verify`.
- **Test abbandonati rimossi:** `test_frr_linkstate.py` , `bench_live_throughput.py` , `bpf_introspect.py` — tentativi di test su traffico reale / guasto reale bloccati da limiti dell'ambiente (il fabric Kathara non consegna UDP:9999 end-to-end; `bpftool` assente dalle immagini). Rimossi per non lasciare in repo test non pienamente verificabili.

Analisi: colli di bottiglia trasversali del datapath

Identificati quattro costi **indipendenti dalla scelta pipeline** (identici in P1/P2/P3, riguardano il framework comune non l'inferenza). Uno è già stato risolto (Task 10), gli altri tre sono documentati come future work:

Collo di bottiglia	Stato
<code>mac_table</code> BPF_HASH con chiavi dense	RISOLTO (Task 10, → ARRAY)
Doppio parse: dispatcher parse tutto per leggere <code>model_id</code> , poi la tail call fa ri-parsare <code>model_<id></code> (i puntatori non sopravvivono alla tail call)	future work
Contatori condivisi: <code>pkt_stats / cls_stats</code> = BPF_ARRAY + <code>__sync_fetch_and_add</code> per pacchetto → cache-line bouncing cross-CPU sotto RSS (dovrebbero essere PERCPU, come già lo scratch di modular)	future work
<code>bpf_redirect(ifindex)</code> invece di <code>bpf_redirect_map()</code> con DEVMAP (flush batch a fine NAPI poll)	future work

Come testare (sul box Linux)

```
# Task 1 – AOT-literal universale
python3 shared/methods/method4_hardcoded_aot.py          # default 65-4-4-7
# + con un model_meta.json custom accanto al .pt per un descrittore sparse

# Task 3 – P2/P3 con IV descrittore-driven (default => output invariato)
sudo python3 shared/execute_pipeline.py --method template --iface eth0 --model-id 0
sudo python3 shared/execute_pipeline.py --method modular  --iface eth0 --model-id 0

# Regressione / verifier / perf
sudo python3 shared/test/test_suite.py --kernel
sudo python3 shared/test/verify_prog_run.py
sudo python3 shared/test/verify_multi_model.py
```

Cosa controllare per primo su Linux. (1) Che i tre programmi **passino il verifier** col nuovo primo layer unrolled (budget complessità). (2) Che l'output del descrittore **default** sia identico a prima (regressione byte-compatibile). (3) Se un descrittore usa `queue_occupancy` , seminare `queue_state` via `queue_state_monitor.init_queue_state()` nel control plane.

Riepilogo in una riga

L'**AOT-literal** non è più bloccato al 65-4-4-7 (P1), e l'**Input Vector** è ora **descrittore-driven a runtime** anche nei map-based P2/P3 (via registry `model_desc`), con il descrittore default byte-compatibile ovunque. Task 2 saltato (non conveniente per design), obsoleti rimossi. Tutto scritto e verificato in Python; **compilazione e verifier da eseguire su Linux**.

Aggiornamenti successivi (Task 5-8, questi **verificati/misurati su Linux**): polling MAC per-classe reso robusto (unica lettura ARP, refresh perenne); AOT-literal ora sostituisce BCC come backend di deploy per hardcoded (richiesta del relatore — loader linkato staticamente contro libbpf, BCC resta solo per il test harness interno); P1 hardcoded ora supporta **profondità arbitraria** (non più fissa a 2 hidden layer); misurato che a parità di budget-pesi **allargare batte approfondire** (2-3× più veloce), coerente su 4 descrittori di feature indipendenti, con un tetto di capacità assoluto ~1200-1300 pesi per l'architettura attuale.